

N - Queens problem in Java :

Solution

```
public class NQueens {
    private int[] queens;
    private int N;

    public NQueens(int N) {
        queens = new int[N];
        this.N = N;
    }

    public void placeNQueens() {
        if (placeQueen(0)) {
            printQueens();
        } else {
            System.out.println("No solution
found");
        }
    }

    private boolean placeQueen(int col) {
        if (col == N) {
            return true; // All queens are
placed successfully
        }

        for (int row = 0; row < N; row++) {
            if (canPlace(row, col)) {
                queens[col] = row; // Place
queen
                if (placeQueen(col + 1)) { //
Recurse; move to next column
```

```

        return true; // Found a
valid placement
    }
    // If no valid placement found,
unplace current queen and backtrack
    }
}

    return false; // No valid placement
found for current column
}

    private boolean canPlace(int row, int col)
{
    for (int i = 0; i < col; i++) {
        if (queens[i] == row ||
Math.abs(queens[i] - row) == Math.abs(i -
col)) {
            return false; // Queen cannot
be placed here
        }
    }
    return true; // It is safe to place the
queen here
}

    private void printQueens() {
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                if (queens[i] == j) {
                    System.out.print("Q ");
                } else {
                    System.out.print("* ");
                }
            }
        }
    }
}

```

```

        System.out.println();
    }
}

public static void main(String[] args) {
    NQueens nQueens = new NQueens(8);
    nQueens.placeNQueens();
}
}

```

Explanation

1. **public class NQueens {** - Declares a public class named NQueens.
2. **private int[] queens;** - Creates a private integer array **queens** to store the positions of the queens on the board.
3. **private int N;** - Declares a private integer N to represent the size of the chessboard.
4. **public NQueens(int N) {** - Defines the constructor of the NQueens class that takes an integer N as input to set the board size.
5. **queens = new int[N];** - Initializes the **queens** array with a size of N.
6. **this.N = N;** - Sets the class's instance variable **N** with the given input N to represent the board's size.
7. **public void placeNQueens() {** - Declares a public method to initiate the process of placing the queens on the board.
8. **if (placeQueen(0)) {** - Calls the **placeQueen** method starting with the first column; if it returns true, it proceeds.
9. **for (int i = 0; i < N; i++) {** - A loop that iterates through each element of the **queens** array to print their positions.

10. `System.out.println("Queen at cell " + i + " is placed at " + queens[i]);` - Prints the position where each queen is placed.
11. `} else {` - Begins the else block, which executes if no solution was found for placing all queens.
12. `System.out.println("No solution found");` - Prints a message indicating that there is no solution.
13. `private boolean placeQueen(int col) {` - Declares a private method that tries to place a queen in the specified column.
14. `if (col == N) {` - Checks if all queens have been placed; if so, returns true.
15. `return true;` - Returns true, indicating that all queens have been successfully placed.
16. `for (int row = 0; row < N; row++) {` - A loop that iterates through each row to find a safe spot for the current queen.
17. `if (isSafe(row, col)) {` - Calls the `isSafe` method to check if it's safe to place a queen at the current row and column.
18. `queens[col] = row;` - If it's safe, places the queen at the current row of the specified column.
19. `if (placeQueen(col + 1)) {` - Recursively calls `placeQueen` for the next column; if it returns true, it continues.
20. `return true;` - If all queens are placed safely, returns true.
21. `queens[col] = 0;` - If it fails to place a queen safely, it resets the current cell to 0 and backtracks.
22. `private boolean isSafe(int row, int col) {` - Declares a private method to check if it's safe to place a queen at the current cell.

23. `for (int i = 0; i < col; i++) {` - A loop that checks each column to the left of the current cell for threats.
24. `if (queens[i] == row || queens[i] - i == row - col || queens[i] + i == row + col) {` - Checks if there's any queen in the same row or diagonals that threatens the current cell.
25. `return false;` - If the current cell is threatened, returns false.
26. `return true;` - If the cell is safe for the queen, returns true.